

# Microservice Architecture Design Document

---

University ERP · Advanced Software Engineering · UCSC

## 1. Executive Summary

This document describes the microservice architecture designed for the University ERP system. The full design decomposes a hypothetical monolithic academic platform into eight independently deployable services, each owning its own MongoDB database. Of these eight, six are implemented and running as a working prototype — Auth, Student, Exam, Result, Transcript, and Notification — while Course Service and Reporting Service remain design-only and are not present in the codebase. Built services communicate via synchronous REST (HTTP/JSON using axios) for real-time queries and asynchronous Kafka events (Apache Kafka 3.7, KRaft mode) for decoupled state propagation; the Kafka pipeline is fully wired and operational, not merely designed. All services are implemented in Node.js with Express and use JWT tokens (issued by the Auth Service) validated independently by each service. The frontend is a single-page application served by NGINX; NGINX itself does not perform API routing in the current build — see Section 8 for the corrected description of the network layer.

## 2. System Context

The University ERP system supports three types of users: students (web browser), administrative staff (Exam Division, HOD, Lecturer, Admin roles via the same web portal), and automated internal processes (Kafka consumers). The system manages the academic lifecycle: student registration and year enrolment, exam scheduling and entry approval, result recording and GPA calculation, transcript generation, and event-driven notifications.

This redesign extracts each business capability into a separate service to achieve independent deployability, fault isolation, and horizontal scalability per domain. It was informed by a gap analysis of an existing Faculty MIS (see supportive-docs/Gap Analysis Report.docx) rather than a purely hypothetical monolith — twelve concrete gaps (G1–G12) were identified and used to prioritise which capabilities to build first.

### 3. Architectural Style — Microservices

The system adopts a microservice architectural style. The key principles applied are:

- Single Responsibility: each service is responsible for exactly one business domain.
- Database per Service: no service accesses another service's database directly.
- API-first: all cross-service communication goes through defined HTTP or event interfaces.
- Loose Coupling: services are unaware of each other's internal implementation.
- High Cohesion: all logic related to a domain lives inside one service.
- Design for Failure: services handle downstream unavailability gracefully (e.g. 502 responses, fail-silent Kafka producers).

#### 3.1 Component Architecture Diagram

The diagram below shows the current state of the system: the six built and running services in full colour, and Course Service / Reporting Service shown greyed out with dashed borders to indicate they are designed but not implemented. This distinction is applied consistently across every diagram in this project (see also [deliverable-docs/architecture-diagram.html](#), the interactive version of this view).

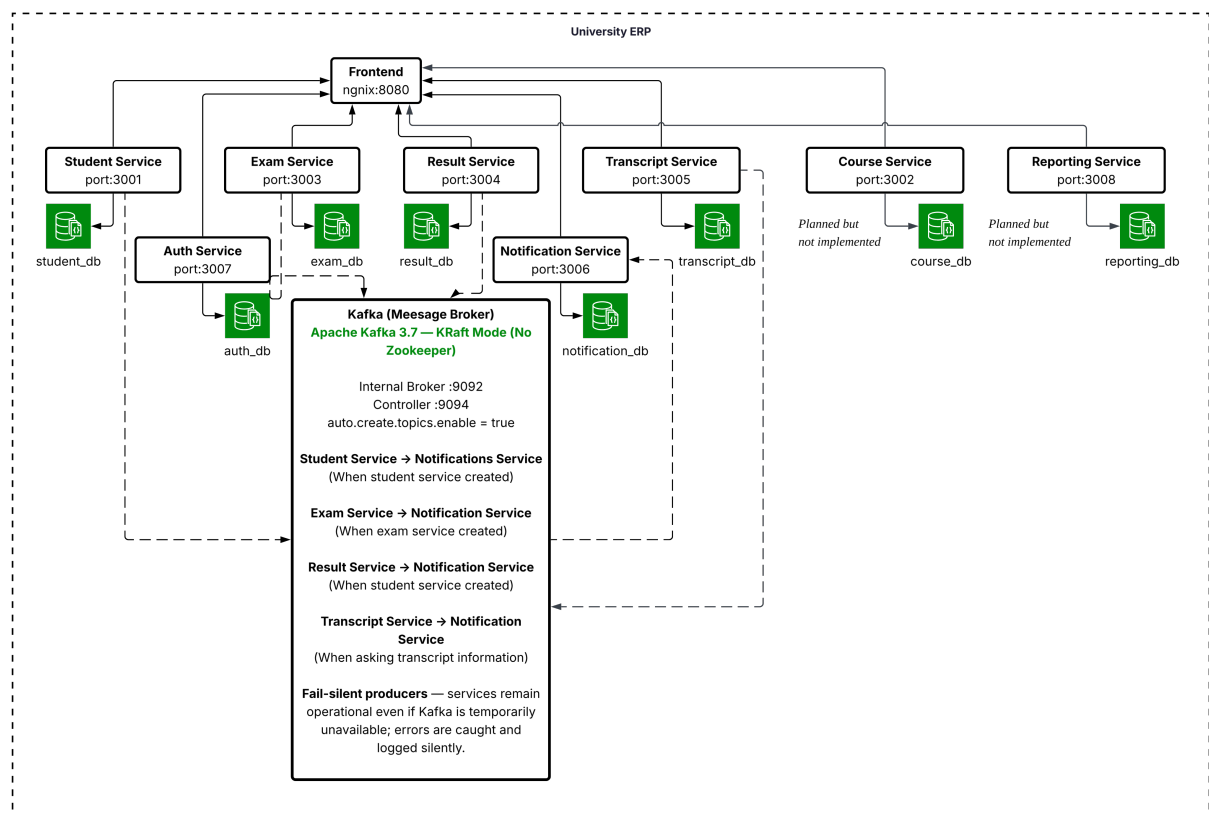


Figure 1 — University ERP component architecture: 6 built services (solid) + 2 planned services (dashed).

## 4. Service Catalogue

All eight designed services are described below. Six are built and running; Course Service and Reporting Service are marked Planned and are not present in services/ or docker-compose.yml. See the Service Identification Table for the full comparison.

### 4.1 Auth Service — Built & Running

|                    |                                                                                                                                                                                                                                                                                                                                   |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Port</b>        | 3007                                                                                                                                                                                                                                                                                                                              |
| <b>Database</b>    | auth_db                                                                                                                                                                                                                                                                                                                           |
| <b>Status</b>      | Built & running                                                                                                                                                                                                                                                                                                                   |
| <b>Description</b> | Issues and validates JSON Web Tokens. All other services validate JWTs independently using the shared JWT_SECRET — no service calls Auth Service at runtime. Auth Service handles user registration, login, demo-user seeding, and bcrypt password management. Supports five roles: STUDENT, LECTURER, HOD, EXAM_DIVISION, ADMIN. |

Actual endpoints:

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/seed — seeds the 5 demo role accounts
- PATCH /api/auth/reset-password

### 4.2 Student Service — Built & Running

|                    |                                                                                                                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Port</b>        | 3001                                                                                                                                                                                                                             |
| <b>Database</b>    | students_db                                                                                                                                                                                                                      |
| <b>Status</b>      | Built & running                                                                                                                                                                                                                  |
| <b>Description</b> | Manages student profiles (35+ fields), year registration, specialization applications, and photo upload. The authoritative source for student identity across the system. Publishes <b>student.created</b> to Kafka on creation. |

Actual endpoints:

- POST /api/students, POST /api/students/bulk (CSV import)
- GET /api/students, GET /api/students/:id
- PATCH /api/students/:id — partial update; immutable fields protected for non-admin roles

- POST /api/students/:id/photo (multer upload)
- POST /api/students/:id/year-registration, GET /api/students/:id/year-registrations, POST /api/students/bulk-year-registration
- Specialization sub-router: GET /api/specializations, POST/GET /api/specializations/:studentId, GET /api/specializations/all/pending, PATCH /api/specializations/:studentId/assign

### 4.3 Course Service — Planned, Not Implemented

|                    |                                                                                                                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Port</b>        | 3002 (reserved, not bound)                                                                                                                                                                |
| <b>Database</b>    | courses_db (not provisioned)                                                                                                                                                              |
| <b>Status</b>      | Planned — no code exists in services/; absent from docker-compose.yml                                                                                                                     |
| <b>Description</b> | Would manage course definitions and modules, providing course metadata consumed by Exam and Transcript services via REST. Remains a design placeholder from the original 8-service scope. |

Designed (not implemented) endpoints:

- POST /api/courses
- GET /api/courses
- GET /api/courses/:id

### 4.4 Exam Service — Built & Running

|                    |                                                                                                                                                                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Port</b>        | 3003                                                                                                                                                                                                                                                                                         |
| <b>Database</b>    | exams_db                                                                                                                                                                                                                                                                                     |
| <b>Status</b>      | Built & running                                                                                                                                                                                                                                                                              |
| <b>Description</b> | Creates exams, manages registration windows, and handles the student exam-entry approval workflow (pending/approved/rejected). Before creating an entry, verifies the student exists via a synchronous REST call to Student Service. Publishes <b>exam.registered</b> to Kafka on enrolment. |

Actual endpoints:

- POST /api/exams, GET /api/exams
- PATCH /api/exams/:id/schedule — set registration window
- PATCH /api/exams/:id/toggle — open/close registration
- POST /api/exams/:id/entries, GET /api/exams/:id/entries
- PATCH /api/exams/:id/entries/:entryId — approve/reject an entry
- GET /api/exams/:id/admission-card — approved entries only

#### 4.5 Result Service — Built & Running

|                    |                                                                                                                                                                                                                                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Port</b>        | 3004                                                                                                                                                                                                                                                                                                    |
| <b>Database</b>    | results_db                                                                                                                                                                                                                                                                                              |
| <b>Status</b>      | Built & running                                                                                                                                                                                                                                                                                         |
| <b>Description</b> | Records subject-level results and computes GPA ( $\Sigma \text{gradePoints} \times \text{credits} / \Sigma \text{credits}$ ) with a grade-to-points mapping (A+/A=4.0 down to F=0.0). GPA records have a draft/final status. Publishes <b>result.published</b> to Kafka when a GPA record is finalised. |

Actual endpoints:

- POST /api/results, POST /api/results/bulk (CSV import)
- GET /api/results, GET /api/results/student/:studentId
- POST /api/results/process-gpa — computes and upserts a draft GPA record
- GET /api/results/gpa/:studentId
- PATCH /api/results/gpa/: gpaId/finalize — sets status=final, emits **result.published**

#### 4.6 Transcript Service — Built & Running

|                    |                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Port</b>        | 3005                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Database</b>    | None — pure aggregator, no owned collections                                                                                                                                                                                                                                                                                                                                 |
| <b>Status</b>      | Built & running                                                                                                                                                                                                                                                                                                                                                              |
| <b>Description</b> | Generates transcript data on demand by aggregating across two other services via synchronous REST: GET /api/students/:id on Student Service (profile) and GET /api/results/... on Result Service (grades and GPA). Returns JSON, not a generated PDF — pdfkit is not used anywhere in this service, despite being listed as a dependency in earlier drafts of this document. |

Actual endpoints:

- GET /api/transcripts/:studentId/semester/:sem?academicYear=X
- GET /api/transcripts/:studentId/final — full academic record, grouped by year then semester

#### 4.7 Notification Service — Built & Running

|                    |                                                                                                                                                                                                                                                                             |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Port</b>        | 3006                                                                                                                                                                                                                                                                        |
| <b>Database</b>    | notif_db                                                                                                                                                                                                                                                                    |
| <b>Status</b>      | Built & running                                                                                                                                                                                                                                                             |
| <b>Description</b> | Pure Kafka consumer. Subscribes to all three topics ( <b>student.created</b> , <b>exam.registered</b> , <b>result.published</b> ), retries subscription every 10s until topics are auto-created, and persists every consumed event as a NotificationLog document for audit. |

Actual endpoints:

- GET /api/notifications — list recent logs, optional ?topic= filter
- GET /api/notifications/student/:studentId

#### 4.8 Reporting Service — Planned, Not Implemented

|                    |                                                                                                                                                                                             |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Port</b>        | 3008 (reserved, not bound)                                                                                                                                                                  |
| <b>Database</b>    | reports_db (not provisioned)                                                                                                                                                                |
| <b>Status</b>      | Planned — no code exists in services/; absent from docker-compose.yml                                                                                                                       |
| <b>Description</b> | Would be a Kafka consumer aggregating metrics for an admin dashboard (pass rates, grade distributions, enrolment counts) from <b>result.published</b> events. Remains a design placeholder. |

Designed (not implemented) endpoints:

- GET /api/reports/summary
- GET /api/reports/exam/:examId

## 5. Technology Decisions

| Technology                              | Used By                                                             | Rationale                                                                                                                                                                                                                             |
|-----------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Node.js 20 (LTS) + Express.js           | All 6 built services                                                | A single language across all services eliminates context-switching. Express is minimal and well-understood. Node's non-blocking I/O suits handling many concurrent HTTP or Kafka messages.                                            |
| MongoDB 7 + Mongoose ODM                | All 6 built services (one logical DB each, single shared container) | MongoDB's document model suits the varied schemas of university data. Mongoose provides application-level schema validation while retaining flexibility. See Section 6.                                                               |
| NGINX (nginx:alpine)                    | Frontend only                                                       | Serves the static single-page frontend on port 8080. In the current build it does not perform API routing — see Section 8 for the corrected description; path-based API gateway routing remains a documented but unbuilt design goal. |
| JSON Web Tokens (jsonwebtoken + bcrypt) | All 6 built services                                                | Stateless authentication — no shared session store required. Each service validates the JWT independently using a shared JWT_SECRET. Tokens are signed for 7 days (see Section 9).                                                    |
| axios                                   | Exam Service, Transcript Service                                    | Promise-based HTTP client for synchronous cross-service REST calls, used only where an immediate response is required (student verification, grade/profile fetch).                                                                    |
| Kafka 3.7 (KRaft mode) + kafkajs        | Student, Exam, Result (producers); Notification (consumer)          | kafkajs is a pure Node.js Kafka client. Running in KRaft mode removes the need for a separate Zookeeper container. Async events decouple producers from consumers. This is fully wired and running, not                               |

|                         |                |                                                                                                                                                                                  |
|-------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         |                | a design-only element.                                                                                                                                                           |
| Docker + docker-compose | Infrastructure | Each built service has a Dockerfile using node:20-alpine. docker-compose.yml wires all containers (MongoDB, Kafka, 6 services, frontend) with one command for local development. |

## 6. Database Strategy — Database-per-Service

Note: This section fulfils Deliverable D6 (Database Strategy), embedded here as permitted by the assignment brief to avoid duplication.

### 6.1 Decision: Database-per-Service

Each of the six built services owns exactly one logical MongoDB database, hosted on a single shared mongodb:7 container in the current Docker Compose setup (a managed cluster such as MongoDB Atlas would be the production equivalent). No service is permitted to issue queries against another service's database — all cross-service data access goes through the owning service's API. The database-per-service \*pattern\* is applied even though, in development, the databases are hosted on one physical MongoDB process rather than eight separate instances; logical isolation, not physical isolation, is the property being enforced.

### 6.2 Why Not a Shared Database?

A single shared database was considered and rejected for the following reasons:

- Schema coupling: changes to a collection used by multiple services require coordinated deployments across those services, negating independent deployability.
- Contention and scaling: a single database becomes a bottleneck; it cannot be scaled independently per service load.
- Failure isolation: a misconfigured query in one service can degrade database performance for all services.
- Ownership ambiguity: with a shared schema, it is unclear which service is responsible for data migrations or schema evolution.

### 6.3 Database Assignments

| Service         | Database    | Key Collections                                      | Status  |
|-----------------|-------------|------------------------------------------------------|---------|
| Auth Service    | auth_db     | User                                                 | Built   |
| Student Service | students_db | Student, YearRegistration, SpecializationApplication | Built   |
| Course Service  | courses_db  | Course, Module (designed only)                       | Planned |
| Exam Service    | exams_db    | Exam, Entry                                          | Built   |



|                      |            |                                            |         |
|----------------------|------------|--------------------------------------------|---------|
| Result Service       | results_db | Result, GPA                                | Built   |
| Transcript Service   | — (none)   | No owned collections — aggregates via REST | Built   |
| Notification Service | notif_db   | NotificationLog                            | Built   |
| Reporting Service    | reports_db | Snapshot, Metric (designed only)           | Planned |

## 6.4 MongoDB Justification

MongoDB was selected over PostgreSQL (which appeared in early architecture drafts) for the following reasons:

- Schema flexibility: different services have vastly different data shapes (e.g. Student's 35+ optional fields). MongoDB's document model avoids migration overhead during iterative development.
- Mongoose ODM: provides schema validation, middleware hooks (pre-save, post-save — used for automatic grade-to-points computation in Result Service), and a familiar query API in Node.js.
- Single cluster, isolated databases: a MongoDB instance natively supports multiple logical databases, enabling the database-per-service pattern without provisioning separate database servers.
- Consistency with the team's existing stack: all services are Node.js; MongoDB is the natural fit in the MERN/MEAN ecosystem.

## 7. Inter-Service Communication

### 7.1 Synchronous REST (HTTP/JSON)

Used when an immediate, consistent response is required before proceeding. The caller blocks until the response is received. axios is used as the HTTP client.

| Caller             | Called          | Endpoint                                                 | When                                                        |
|--------------------|-----------------|----------------------------------------------------------|-------------------------------------------------------------|
| Exam Service       | Student Service | GET /api/students/:id                                    | Before creating an exam entry — verifies the student exists |
| Transcript Service | Student Service | GET /api/students/:id                                    | When building a transcript — fetches student profile        |
| Transcript Service | Result Service  | GET /api/results?... and GET /api/results/gpa/:studentId | When building a transcript — fetches results and GPA        |

## 7.2 Asynchronous Kafka Events

Used for state propagation that does not require an immediate response. This is fully implemented and running against Apache Kafka 3.7 in KRaft mode — producers are fail-silent (a service stays up and simply skips publishing if Kafka is unreachable), and the Notification Service consumer retries topic subscription every 10 seconds until topics are auto-created by the first publish.

| Event                   | Producer        | Consumers            | Trigger                                |
|-------------------------|-----------------|----------------------|----------------------------------------|
| <b>student.created</b>  | Student Service | Notification Service | Successful POST /api/students          |
| <b>exam.registered</b>  | Exam Service    | Notification Service | Successful POST /api/exams/:id/entries |
| <b>result.published</b> | Result Service  | Notification Service | PATCH /api/results/gpa/:gpaId/finalize |

Note: Reporting Service and Transcript Service are not currently Kafka consumers of **result.published** — only Notification Service consumes all three topics in the built system. A future Reporting Service would be a natural fourth consumer of **result.published**.

## 8. Network Layer — Actual vs. Designed Routing

### 8.1 As-Built: Direct Service Ports (Current State)

In the current build, NGINX (nginx:alpine, docker-compose service "frontend") serves only the static single-page frontend on port 8080 and performs no API routing — its nginx.conf contains a single location block with try\_files for the SPA and a caching rule for JS/CSS. There is no path-based reverse proxy in front of the backend services.

Each of the six built services publishes its port directly to the host via docker-compose.yml (3007, 3001, 3003, 3004, 3005, 3006), and the frontend JavaScript (frontend/js/app.js) calls each service directly at http://localhost:<port> rather than through a shared gateway path. This means the "NGINX is the sole external entry point" and "service ports are not exposed to the public network" claims made in earlier drafts of this document and in Benefits & Risks §3.6 do not hold for the current build and should be read as a stated production goal, not a description of the running system.

### 8.2 Designed: Path-Based API Gateway (Not Yet Implemented)

The table below is the original design intent for NGINX as a true API gateway. It has not been implemented — it is retained here as the target design for a future iteration.

| Path Prefix   | Upstream Service | Port | Status                         |
|---------------|------------------|------|--------------------------------|
| /api/auth     | Auth Service     | 3007 | Designed, not routed via NGINX |
| /api/students | Student Service  | 3001 | Designed, not routed via NGINX |

|                    |                      |      |                                    |
|--------------------|----------------------|------|------------------------------------|
| /api/courses       | Course Service       | 3002 | Designed; service itself not built |
| /api/exams         | Exam Service         | 3003 | Designed, not routed via NGINX     |
| /api/results       | Result Service       | 3004 | Designed, not routed via NGINX     |
| /api/transcripts   | Transcript Service   | 3005 | Designed, not routed via NGINX     |
| /api/notifications | Notification Service | 3006 | Designed, not routed via NGINX     |
| /api/reports       | Reporting Service    | 3008 | Designed; service itself not built |

## 9. Authentication Strategy

JWT (JSON Web Tokens) are used for stateless authentication. The Auth Service issues a signed JWT on login, containing a role claim. The client includes the token in the Authorization: Bearer <token> header on every subsequent request. Each service independently validates the token using the shared JWT\_SECRET environment variable — no runtime call to Auth Service is needed per request.

Five roles are supported: STUDENT, LECTURER, HOD, EXAM\_DIVISION, and ADMIN. Tokens are signed for 7 days (jwt.sign(..., { expiresIn: '7d' }) in services/auth-service/src/routes/auth.js) — this corrects earlier drafts of this document, which stated a 24-hour default. bcrypt is used for password hashing with a work factor of 12. Demo accounts for all five roles can be created via POST /api/auth/seed.

## 10. Deployment Overview

All six built services are containerised using Docker with node:20-alpine base images. docker-compose.yml defines the full local topology:

- mongodb — single mongo:7 container, port 27017, isolated volume (mongo-data); hosts all six logical databases
- kafka — apache/kafka:3.7.0, KRaft mode (no Zookeeper), broker on 9092 (internal) / 9094 (external), isolated volume (kafka-data), auto.create.topics.enable=true
- auth-service — port 3007 · student-service — port 3001 · exam-service — port 3003 · result-service — port 3004 · transcript-service — port 3005 · notification-service — port 3006
- student-service additionally mounts uploads-data for profile photo storage
- frontend — nginx:alpine serving the static SPA, port 8080:80
- All services share the erp-network Docker bridge for inter-service DNS resolution (e.g. http://student-service:3001)

- JWT\_SECRET injected via .env file at runtime (not hardcoded); .env is gitignored, only .env.example is committed

Six of the eight designed services are built and running: Auth, Student, Exam, Result, Transcript, and Notification. Course Service and Reporting Service remain design-only — they have no directory under services/, no Dockerfile, and no entry in docker-compose.yml.

## 11. Assumptions and Constraints

- Course Service and Reporting Service are designed in this document (Sections 4.3, 4.8) but not implemented; all endpoint lists, database assignments, and diagrams for these two are aspirational.
- Kafka is fully implemented and running in the current build (contrary to earlier drafts of this document) — **student.created**, **exam.registered**, and **result.published** are live topics consumed by Notification Service.
- NGINX does not currently perform path-based API routing (Section 8); each service's port is exposed directly on the host and called directly by the frontend. Path-based gateway routing remains a documented but unbuilt design goal.
- JWT\_SECRET is shared across all services via a single .env file; in production this would be managed by a secrets vault.
- MongoDB runs as a single shared container in development (one process, multiple logical databases). A managed multi-database cluster (e.g. MongoDB Atlas) is the assumed production equivalent.
- Transcript Service returns JSON, not a generated PDF — pdfkit is not used despite appearing as a dependency in earlier drafts of this document.
- No automated tests exist in the repository for any service at the time of writing (no \*.test.js/\*.spec.js files, no test script in any package.json).
- No rate limiting or request throttling is implemented in any service.

References: Service Identification Table, Bounded Context Diagram, API Endpoint Design, Service Communication Diagram, Benefits & Risks.